StudyForge AI

Complete Theoretical Explanation

Simple guide with extra focus on the React frontend

Project purpose	AI learning platform for notes, summaries, flashcards, quizzes, and progress.
Technology stack	React + FastAPI + SQLite + SQLAlchemy + Pydantic + JWT + Gemini
Frontend status	Login/signup interface exists; dashboard features are the next stage.

1. Project Overview

StudyForge AI helps students create accounts, organize subjects, save notes, upload PDF/TXT material, generate AI study content, take quizzes, and track scores. The frontend is the visible workspace; the backend validates requests, protects data, stores records, calls Gemini, and calculates results.

```
Student action -> React frontend -> FastAPI API -> Security -> Database/Gemini -> JSON -> React screen
```

Main journey

- Create an account and log in.
- Create a subject such as Python or DBMS.
- Add notes or upload a file.
- Generate a summary, flashcards, or quiz.
- Submit a quiz and receive a percentage.
- Review learning progress.

2. Technologies

Technology	Meaning	Use
Python	Programming language	Backend
FastAPI	Web framework	API endpoints
Uvicorn	Web server	Runs FastAPI
SQLite	Local database	Stores data
SQLAlchemy	Database helper	Python-to-table mapping
Pydantic	Data checker	Validation
JWT + Bcrypt	Token + password hash	Authentication
Gemini AI	AI model	Study content
React	UI library	Frontend
CORS	Browser permission	Frontend/backend access
pypdf	PDF reader	Text extraction

3. Frontend Architecture

The current frontend is in frontend/src/App.jsx. It renders a login/signup card and calls FastAPI. A larger version should divide the UI into reusable components and pages.

```
frontend/src/
  App.jsx                # main component
  components/            # Navbar, cards, quiz question
  pages/                 # Login, Dashboard, Subjects, Quiz, Progress
  services/api.js        # fetch helpers
  styles/                # CSS
```

React components

A component is a reusable UI block. App is the current main component. Future components could be LoginForm, SubjectCard, MaterialCard, QuizQuestion, and ProgressDashboard.

JSX

JSX lets JavaScript contain HTML-like markup. Use curly brackets for JavaScript expressions and camelCase events such as onClick, onChange, and onSubmit.

```
<main>
  <h1>StudyForge AI</h1>
  <p>Learn smarter with AI-powered study tools.</p>
</main>
```

4. React State, Inputs, and Events

State stores information that can change while the page is open. When state changes, React redraws the affected UI.

```
const [isSignup, setIsSignup] = useState(false);
const [form, setForm] = useState({ name: '', email: '', password: '' });
const [message, setMessage] = useState('');
```

State	Purpose
isSignup	Chooses login or signup mode
form	Stores input values
message	Shows success/error text

Controlled input

The input value comes from React state. onChange updates the state, so React always knows what the user typed.

```
function updateForm(event) {
  setForm({ ...form, [event.target.name]: event.target.value });
}
```

Conditional rendering shows an element only when a condition is true.

```
{isSignup && <input name="name" placeholder="Your name" />}
```

5. Authentication and API Calls

The same form supports both modes. The endpoint changes according to isSignup.

```
const endpoint = isSignup ? '/auth/signup' : '/auth/login';
```

handleSubmit prevents a normal browser refresh, sends a POST request, and reads the JSON result. async/await lets the frontend wait for the server.

```
async function handleSubmit(event) {
  event.preventDefault();
  const response = await fetch(`http://127.0.0.1:8000${endpoint}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(form),
  });
  const data = await response.json();
}
```

After login, the JWT is saved and sent with protected requests.

```
localStorage.setItem('token', data.access_token);

headers: {
  Authorization: `Bearer ${localStorage.getItem('token')}`,
}
```

If response.ok is false, the frontend displays data.detail or a friendly error. try/catch should handle a server that cannot be reached.

6. Frontend Screens

Screen	Responsibility
Login/Signup	Create account and log in
Dashboard	Show welcome, subject count, scores, and progress
Subjects	Create, list, open, and delete subjects
Materials	Write notes, upload files, and view material
AI Tools	Request summaries, flashcards, and quizzes
Quiz	Show questions, record answers, submit, show score
Progress	Show averages, best score, and recent attempts

Loading and lists

```
1. Page loads.
2. useEffect calls an API.
3. JSON is saved with setState.
4. React renders cards/lists.
5. User action sends another request.
6. State refreshes the screen.
```

```
{subjects.map((subject) => (
  <div key={subject.id}>
    <h3>{subject.name}</h3>
  </div>
))}
```

map repeats the UI for every item. key helps React track each item. A loading state such as Generating quiz... is important for AI requests.

File upload

```
const formData = new FormData();
formData.append('subject_id', subjectId);
formData.append('title', title);
formData.append('file', selectedFile);
fetch('/uploads/material', { method: 'POST', body: formData });
```

With FormData, do not manually set Content-Type; the browser supplies the multipart boundary.

7. Quiz and Progress Frontend

The quiz screen stores the selected answer for each question and sends all answers when the student submits.

```
const [answers, setAnswers] = useState({});
setAnswers({ ...answers, [question.id]: selectedOption });
```

The backend compares submitted and correct answers, calculates the percentage, saves the attempt, and returns the result. The progress screen displays quizzes taken, average score, best score, correct answers, total questions, and recent attempts.

8. Backend Files and Functions

File	Purpose
main.py	Creates FastAPI app, CORS, tables, and routers
database.py	SQLite engine, sessions, Base, get_db()
models.py	User, Subject, Material, Quiz, Question, Attempt tables
schemas.py	API input/output formats and validation
Authentication.py	signup() hashes; login() verifies and creates token
security.py	create_access_token() and get_current_user()
subjects.py	Subject create/list/delete
materials.py	Ownership check and note endpoints
uploads.py	PDF/TXT validation and text extraction
ai_service.py	Gemini summary, flashcard, quiz functions
ai_routes.py	AI endpoints and quiz saving
quizzes.py	Get quiz and score submission
progress.py	Aggregate statistics and recent attempts

9. Database, Security, and API Theory

```
User -> Subject -> StudyMaterial -> Quiz -> QuizQuestion
User -> QuizAttempt
```

Models describe database storage. Schemas describe API data. Foreign keys connect records. Every protected query checks ownership so users cannot access other users data. Passwords are Bcrypt hashes, not plain text. JWT contains the user identity and expiry time. The default development SECRET_KEY must be replaced before deployment.

Main endpoints

Method	Endpoint	Purpose
GET	/	Welcome
GET	/health	Health check
POST	/auth/signup	Create account
POST	/auth/login	Receive token
POST	/subjects/	Create subject
GET	/subjects/	List subjects
DELETE	/subjects/{id}	Delete subject
POST	/materials/	Add note
GET	/materials/subject/{id}	List notes
POST	/uploads/material	Upload file
POST	/ai/summary	AI summary
POST	/ai/flashcards	AI flashcards
POST	/ai/quiz	Generate/save quiz
GET	/quizzes/{id}	View quiz
POST	/quizzes/{id}/submit	Submit quiz
GET	/progress/overview	Progress

10. Complete Request Flow

```
1. Student selects a note in React.
2. React sends POST /ai/quiz with material ID.
3. FastAPI reads the JWT.
4. User identity and ownership are checked.
5. Gemini creates structured questions.
6. Backend saves quiz records.
7. FastAPI returns JSON.
8. React renders the quiz.
```

11. Important Concepts for Viva

Concept	Simple explanation
useState	Stores changing React data
useEffect	Loads data or runs side effects
fetch	Sends browser requests
async/await	Waits for asynchronous work
map()	Renders repeated UI
event.preventDefault()	Stops form refresh
Depends()	Provides FastAPI dependencies
db.commit()	Saves database changes
join/filter	Connects/selects records
try/except	Handles errors

Final mental model

React displays the learning interface, fetch connects it to FastAPI, JWT identifies the student, SQLAlchemy stores learning data, and Gemini creates AI study content. The current frontend has login/signup; the next major work is the dashboard, subjects, materials, AI tools, quizzes, and progress pages.

```
py -3.10 -m pip install -r requirements.txt
py -3.10 -m uvicorn main:app --reload
API docs: http://127.0.0.1:8000/docs
```